

# NOC Alarm Intelligence

Dashboard — Project Documentation & README

[How it works](#) · [Technology stack](#) · [Analytics & ML models](#) · [Setup](#)

DTS Wireless — Network Operations Center

Document version 1.0 · Generated 2026-06-25

Primary application: **DASHBOARD/alarm\_dashboard\_4.py**

# 1. Project Overview

The **NOC Alarm Intelligence Dashboard** is a browser-based analytics application for Network Operations Center (NOC) alarm data. NOC engineers export alarm *EventLog* reports from the alarm-management system as Excel files; the dashboard ingests one or many of these files and turns them into interactive analytics, device-health scoring, temporal pattern analysis, predictive insight, a trainable machine-learning forecast, multi-file comparison, and editable PowerPoint / CSV reports.

It runs entirely on the analyst's machine (no server or database required) and is built as a single Python script using the Streamlit framework. Everything is computed live from the uploaded files.

## At a glance

- **Input:** Excel alarm exports (.xls / .xlsx), sheet named *EventLog*.
- **Output:** interactive charts, health scores, forecasts, PowerPoint & CSV downloads.
- **Runtime:** 100% local — Python + Streamlit in the browser.
- **Audience:** NOC engineers and operations managers.

# 2. Key Features (Tabs)

| Tab            | What it does                                                                                                                                                     |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Overview       | Headline KPIs, alarms by Level and by BIU, and a daily alarm timeline.                                                                                           |
| Alarm Analysis | Level pie, duration box-plots, Device×BIU and Level×BIU heatmaps, a Level→BIU→Device→Name sunburst, top names/messages, and PowerPoint export.                   |
| Device Health  | Per-device Health Score (0–100) with MTBF, MTTR, a reliability scatter map, and a downloadable health table.                                                     |
| Time Patterns  | Alarms by hour, by day-of-week, a Day×Hour peak heatmap, severity-by-hour, and per-BIU daily trend.                                                              |
| Predictions    | Statistical next-alarm estimate and urgency bucket per device (live statistic).                                                                                  |
| ML Forecast    | A trained Gradient Boosting model that forecasts upcoming daily alarm volume; supports adding more historical files to improve accuracy and exporting the model. |
| Compare Files  | Side-by-side comparison of multiple uploaded files (totals, levels, devices, BIUs, health, daily overlay).                                                       |

# 3. Technology Stack

## Language

**Python 3.13.5** — the entire application is written in Python; no other programming language is used. The UI is declared in Python via Streamlit (no separate HTML/JS/CSS files, although small inline CSS snippets style the theme).

## Libraries & tools

| Component          | Technology      | Version | Purpose in this project                                                   |
|--------------------|-----------------|---------|---------------------------------------------------------------------------|
| UI framework       | Streamlit       | 1.55.0  | Renders the whole web app, widgets, tabs, file uploads, caching.          |
| Data engine        | pandas          | 2.3.3   | Load, clean, group, pivot and aggregate the alarm tables.                 |
| Numerics           | NumPy           | 2.3.1   | Vectorised math used in scoring and the ML feature pipeline.              |
| Charts             | Plotly          | 6.5.1   | All interactive charts (bar, pie, box, line, heatmap, sunburst, scatter). |
| Machine learning   | scikit-learn    | 1.7.2   | Gradient Boosting forecaster, train/test metrics.                         |
| Model I/O          | joblib          | 1.5.2   | Serialises the trained model for download/reuse.                          |
| Excel (.xls/.xlsx) | python-calamine | 0.6.2   | Fast, robust primary engine; reads many corrupted .xls files.             |
| Excel (.xlsx)      | openpyxl        | 3.1.5   | Fallback reader for modern .xlsx workbooks.                               |
| Excel (.xls)       | xlrd            | 2.0.2   | Fallback reader for legacy .xls workbooks.                                |
| PowerPoint         | python-pptx     | 1.0.2   | Generates the editable .pptx report with native charts.                   |
| Image export       | kaleido         | 1.2.0   | Renders Plotly charts to PNG for download.                                |
| PDF (this doc)     | reportlab       | 5.0.0   | Generates this documentation PDF.                                         |

Versions reflect the environment in which this document was generated; the app also degrades gracefully if an optional package (e.g. scikit-learn or kaleido) is missing.

## 4. How It Works — Data Pipeline

Data flows through six stages every time the app runs:

| Stage      | Function / location          | Detail                                                                                                   |
|------------|------------------------------|----------------------------------------------------------------------------------------------------------|
| 1. Upload  | st.file_uploader (sidebar)   | One or more .xls/.xlsx files; an optional .pptx template.                                                |
| 2. Read    | get_sheets() / read_sheet()  | Tries calamine first, then openpyxl (.xlsx) or xlrd (.xls), then an xlrd corruption-tolerant fallback.   |
| 3. Clean   | clean_df()                   | Drops No./Type/Note; parses timestamps; derives Duration, Severity, Device Short, Hour, DayOfWeek, Date. |
| 4. Filter  | sidebar widgets              | Optional Level / BIU / date-range filters applied to the analysis.                                       |
| 5. Analyse | the 7 tabs                   | Charts, health scoring, predictions, ML forecast, comparison.                                            |
| 6. Export  | build_ppt(), download_button | Editable PowerPoint, CSV tables, PNG charts, trained model.                                              |

### Robust Excel reading

Alarm-system exports are often slightly malformed OLE2 files that strict readers reject. The loader therefore cascades through engines and, as a last resort, opens the workbook with xlrd in corruption-tolerant mode and rebuilds the sheet cell-by-cell. python-calamine is auto-installed on first run if missing.

### Cleaning rules (clean\_df)

- Drops the non-analytical columns No., Type, Note.
- Parses **Time Received** / **Time Cleared** to datetimes.
- **Duration (Min)** = (Time Cleared – Time Received) in minutes, clipped at  $\geq 0$ .
- Fills blank Level / BIU / Device / Name / Message with 'Unknown'.
- **Device Short** = last dash-separated segment of the device path (e.g. ODU#1-DOU2-eHUB#1-eRDU2 → eRDU2).
- **Severity** mapped from Level; **Hour**, **DayOfWeek**, **Date** derived from Time Received.

## 5. Expected Data Format

The dashboard expects an alarm **EventLog** sheet with the following columns (device-inventory exports, which lack these, are detected and reported):

| Column | Type | Used for                                      |
|--------|------|-----------------------------------------------|
| No.    | int  | Row index (dropped during cleaning).          |
| Type   | text | Record type (dropped).                        |
| Level  | text | Severity: Critical / Major / Minor / Warning. |

| Column        | Type     | Used for                                               |
|---------------|----------|--------------------------------------------------------|
| BIU           | text     | Base Interface Unit identifier.                        |
| Device        | text     | Full device path.                                      |
| Name          | text     | Alarm / equipment name.                                |
| Message       | text     | Alarm message text.                                    |
| Time Received | datetime | When the alarm was raised (core of all time analysis). |
| Time Cleared  | datetime | When the alarm cleared (drives Duration).              |
| Note          | text     | Free notes (dropped).                                  |

## 6. Analytics & Models

### 6.1 Severity mapping

Each alarm Level is converted to a numeric severity used in scoring:

| Level  | Critical | Major | Minor | Warning | Informational |
|--------|----------|-------|-------|---------|---------------|
| Weight | 4        | 3     | 2     | 1       | 0             |

### 6.2 Device Health Score (0–100)

A composite, normalised penalty score computed per device. Each input is min–max normalised across devices, then combined into a weighted penalty; the score is the inverse of that penalty:

```
penalty = 0.35·frequency + 0.35·severity + 0.15·MTTR + 0.15·(1 - MTBF)
Health Score = 100 × (1 - penalty) [clipped 0–100]
```

- **Frequency** = alarms per day; **Severity** = mean numeric severity.
- **MTTR** (Mean Time To Repair) = average alarm Duration in minutes.
- **MTBF** (Mean Time Between Failures) = average gap between a device's alarms, in hours (higher MTBF lowers the penalty).
- Status bands: ≥80 Good · 60–79 Fair · 40–59 Poor · <40 Critical.

### 6.3 Statistical Prediction Engine (Predictions tab)

This is **not** machine learning and requires no training — it is a live statistic recomputed from the data on every run. For each device with at least two alarms:

- Compute **inter-arrival times** (hours between consecutive alarms).
- Take an **exponentially-weighted mean (EWM)** of those gaps (span = min(3, n)); recent gaps weigh more, so the estimate adapts to a changing rate.
- **Predicted next alarm** = last alarm time + EWM gap.
- **Confidence %** =  $100 \times (1 - CV)$ , where  $CV = \text{std} \div \text{mean}$  of the gaps (coefficient of variation). Regular pattern → high confidence.
- **Reference time** for urgency defaults to the *latest alarm in the data* (so historical exports are not all flagged 'overdue'); a toggle allows wall-clock 'Now'.

### 6.4 Machine-Learning Forecast (ML Forecast tab)

A trainable model that forecasts **daily alarm volume** for the upcoming days. This is the component that genuinely 'learns' from data and improves with more history.

| Aspect    | Detail                                                                                                       |
|-----------|--------------------------------------------------------------------------------------------------------------|
| Algorithm | scikit-learn GradientBoostingRegressor (n_estimators=200, max_depth=3, learning_rate=0.05, random_state=42). |
| Target    | Number of alarms per calendar day (continuous series, gaps filled with 0).                                   |

| Aspect       | Detail                                                                                     |
|--------------|--------------------------------------------------------------------------------------------|
| Features     | day-of-week, day-of-month, month, is_weekend, lag-1, lag-7, rolling 3-day and 7-day means. |
| Train / test | Time-ordered split; the last $\leq 14$ days are held out for honest evaluation.            |
| Metrics      | MAE, RMSE, $R^2$ on the holdout, plus an actual-vs-predicted chart.                        |
| Forecast     | Recursive multi-step (7–30 day horizon), predictions clipped at $\geq 0$ .                 |
| Export       | Trained model downloadable as .joblib; forecast and combined history as CSV.               |
| Data guard   | Requires $\geq 21$ days of history; otherwise a clear message is shown.                    |

### Adding more history (improving accuracy)

The ML Forecast tab includes a training-data uploader. It trains on the current sheet's full history plus any additional EventLog files you add, concatenated and **de-duplicated** (on Time Received + Device + Name + Message + Level + BIU). Accuracy improves as the *training window in days* grows — so add files covering **new, earlier/later date ranges for the same site**, not re-exports of the same period. A 'combined history' CSV can be downloaded and re-uploaded later to accumulate history across sessions.

## 7. Installation & Running

Install the dependencies (one time):

```
pip install streamlit pandas numpy plotly scikit-learn python-pptx \
    openpyxl xlrd python-calamine kaleido joblib
```

Launch the application (note: a Streamlit app must be started with `streamlit run`, never plain python):

```
python -m streamlit run "DASHBOARD/alarm_dashboard_4.py"
```

The app opens at <http://localhost:8501>. Upload your EventLog Excel file(s) in the sidebar and use the tabs. Press Ctrl+C in the terminal to stop.

## 8. Project Structure

| Path                                  | Description                                                                               |
|---------------------------------------|-------------------------------------------------------------------------------------------|
| DASHBOARD/alarm_dashboard_4.py        | Main application — the 7-tab NOC Alarm Intelligence Dashboard (subject of this document). |
| DASHBOARD/Alarm Analysis Dashboard.py | Simpler earlier dashboard (analysis + PowerPoint export).                                 |
| DASHBOARD/Program data/*.xls          | Sample alarm EventLog exports (DMS, Peabody, Waltham, Needham, ...).                      |
| DASHBOARD/generate_readme_pdf.py      | This document's generator script.                                                         |
| (project root) *.xls / *.xlsx         | Device-inventory exports (BIU ID, Name, Firmware ...) — not alarm data.                   |

## 9. Outputs & Exports

- **PowerPoint (.pptx)** — editable report with native, in-PowerPoint charts (python-pptx); supports an optional uploaded template.
- **CSV** — device-health table, predictions, ML forecast, and combined training history.
- **PNG** — individual charts via the download buttons (requires kaleido).
- **Model (.joblib)** — the trained forecasting model for reuse.

## 10. Limitations & Tips

- **ML accuracy needs history:** daily-volume forecasting needs continuous, same-site history (aim for 90+ days). With only a few weeks,  $R^2$  can be negative — this is expected.
- **Alarm data is bursty:** mostly-quiet days with occasional spikes make exact volume prediction hard; treat forecasts as indicative.
- **One site per model:** combining different sites trains a blurred 'whole-network' model.



- **Right file type:** upload the EventLog alarm export, not the device-inventory export.
- **Statistical vs ML:** the Predictions tab is a live statistic (no training); the ML Forecast tab is the trained, improvable model.

## 11. Glossary

| Term              | Meaning                                                                         |
|-------------------|---------------------------------------------------------------------------------|
| NOC               | Network Operations Center.                                                      |
| BIU               | Base Interface Unit — a monitored hardware unit.                                |
| MTBF              | Mean Time Between Failures — average gap between a device's alarms (hours).     |
| MTTR              | Mean Time To Repair — average time an alarm stays active (minutes).             |
| EWM               | Exponentially-Weighted Mean — average that weights recent values more.          |
| CV                | Coefficient of Variation — $\text{std} \div \text{mean}$ ; measures regularity. |
| EventLog          | The alarm-event sheet exported by the alarm-management system.                  |
| Gradient Boosting | An ensemble ML algorithm that builds many small decision trees sequentially.    |

End of document — NOC Alarm Intelligence Dashboard, DTS Wireless NOC.